The handl kernel: a closed arrow calculus for composable languages

Draft — proposed, not accepted

handl project

2026-10-21

Abstract

A language toolchain that must understand every construct everywhere — parser, evaluator, typechecker, rewriter, serialiser — collapses under its own surface area. We describe the handl kernel, the layer of the handl platform that is deliberately not a language: a closed six-combinator arrow calculus (`arr`, `>>>`, `first`, `choice`, `loop`, `app`) realised as the `Term` datatype, a canonical tagged `Value` representation in which arrows are self-contained serializable data, and the `Language` schema — a pure-data description of tokens, grammar, AST shapes, eval schemes, type rules, and transforms that generic engines dispatch on. Effects enter through the Graded Modal Sanada Arrow Calculus: a grade row on the arrow type is the modality, an operation request is a graded `Op`, and a handler is an ordinary arrow value whose installation eliminates the grade. The same dimension slot that carries a named grade carries a numeric dimension, so programs are 1-morphisms, laws and rewrites are 2-morphisms, and `>>>[k]` pastes cells along the k-th globular boundary — one mechanism from computation to law. The kernel is closed: features enter by desugaring, normalisation, or handler interpretation, never by new combinators; what is not in the kernel — syntax, modules, rule languages, closures, engines, host composition — is named explicitly.

> **!** Important
>
> **Status.** This is a working draft — *proposed, not accepted*. It has not had human revision or external review. It is the kernel paper: it owns the calculus narrative once so the platform and domain papers can cite it instead of repeating it. Every claim about the implementation names the crate, module, or file that carries it; the repository is `handl-lang/kernel`.

1 Introduction

A programming language is usually specified as a list of features, and every consumer of the language — the parser, each interpreter, the typechecker, the serialiser, the pretty-printer, every analysis — must know all of them. The coupling is multiplicative: each new construct is a new obligation on every consumer, and each consumer is a new place for two implementations of the same construct to disagree.

The handl platform inverts that pressure twice, at two different levels, and the inversion is the subject of this paper:

1. **The calculus is closed.** Everything that executes is a `Term` in a six-combinator arrow calculus — Hughes' arrows (Hughes 2000) in the Lindley–Wadler–Yallop formulation (Lindley, Wadler, and Yallop 2010), extended with operations and handlers in Sanada's style (Sanada 2024). New language features do not add new combinators; they are expressed as combinations, desugarings, or normalisations of the existing set.
2. **A language is data.** The tokens, grammar, AST shapes, eval schemes, type rules, and transforms that define a language are ordinary serializable values — fragment tables merged into a `Language` object — not code compiled into an engine. Adding a language construct is a data change; adding a calculus combinator is an engine edit; the two almost never happen together.

Together these give a graded algebraic theory in the sense of Plotkin–Power (G. Plotkin and Power 2003), Katsumata (Katsumata 2014), and Kura (Kura 2020): a `Language` is a signature, a `Term` is a term in the free arrow category over that signature, and an execution engine is a *model* of the theory — the Lindenbaum–Tarski reading in which syntax is data and semantics is a choice of interpretation. Because arrows rather than functions are primitive, the model layer is plural by construction: tree-walk, bytecode, and native-code engines interpret the same `Term`s, and the type-and-law machinery interprets them again, differently.

This paper is deliberately scoped to the kernel. It does not describe the surface language (still unreleased), the engine implementations (one repository per operation, `handl-lang/engine*`), or the platform layer that composes languages, kits, and plugins (`handl-lang/host`, whose own account belongs to the platform paper). It describes the shared foundation those layers consume:

- Section 2 — the `Term` kernel: six combinators, the `Op`/`Handle` effect pair, and the honest accounting of the remaining auxiliary constructors;
- Section 3 — the `Value`/`TaggedValue` representation, including arrows as serializable data and nominal identity;
- Section 4 — the `Language` schema and its merge semantics;
- Section 5 — the Graded Modal Sanada Arrow Calculus: grades as modality, operations as requests, handlers as elimination;
- Section 6 — the dimension/axis structure: programs, laws, and higher transformations as n-morphisms of one mechanism;
- Section 7 — why the set is closed, and the criterion for growth;
- Section 8 — what is deliberately not in the kernel.

The design claim is narrow: *a small closed calculus plus a data schema is sufficient to host a family of languages without any engine knowing any one of them.* The evidence is the repository itself — the schema, the calculus, and the law machinery are running code, and the remaining gaps are enumerated as approximations in Section 9, not elided.

2 The `Term` kernel

`Term` is the arrow-calculus kernel. It is defined once, in `hnl-engine` (`handl-lang/engine`), and re-exported by `handl-language` (`crates/handl-language/src/term.rs`): a `Term` is an engine-agnostic description of *what* to evaluate, not *how*. Engines interpret terms; the term itself is pure data — a serde-serializable enum. Versioning lives on the `Language`, not the term: `Language::scheme_version` (`SchemeVersion`, canonical `"v2"`) declares which combinator scheme version the language's terms target, and engines declare which versions they support.

2.1 The six combinators

Table 1: The closed combinator set.

Combinator	`Term` variant	Constructs
`arr`	`Arr { dim, value }`	lift a pure value/function into an arrow
`>>>` (`seq`)	`Seq { axis, fst, snd }`	composition along axis `k`
`first`	`First { inner }`	run on the first of a pair
`choice`	`Choice { l, r }`	route on `Either`: left to `l`, right to `r`
`loop`	`Loop { body }`	feedback (trace), not iteration
`app`	`App { callee, args }`	higher-order arrow application

These are the only arrow formers the kernel knows about. With `op` and `handle` (Section 5) they are the classical six combinators of Hughes' arrow interface — a set that spans four typeclasses: `arr`/`>>>` give a category (`Arrow`), `first` gives product structure, `choice` sum routing (`ArrowChoice`), `loop` a trace (`ArrowLoop`), and `app` closedness — the `ArrowApply` exponential that recovers monadic power, `ArrowApply` $\cong$ `Monad` (Hughes 2000), with the equational correspondence carried out in the arrow calculus itself by Lindley–Wadler–Yallop (Lindley, Wadler, and Yallop 2010).

The typing rules, at the theory level, are the usual ones. Writing $f : A \to B$ for a pure function and $a : A \rightsquigarrow B$ for an arrow:

$$\frac{f : A \to B}{\mathrm{arr}\, f : A \rightsquigarrow B} \qquad \frac{a : A \rightsquigarrow B \quad b : B \rightsquigarrow C}{a \ggg b : A \rightsquigarrow C}$$

$$\frac{a : A \rightsquigarrow B}{\mathrm{first}\, a : (A \times C) \rightsquigarrow (B \times C)} \qquad \frac{a : A \rightsquigarrow C \quad b : B \rightsquigarrow C}{\mathrm{choice}\, a\, b : (A + B) \rightsquigarrow C}$$

$$\frac{a : (A \times C) \rightsquigarrow (B \times C)}{\mathrm{loop}\, a : A \rightsquigarrow B} \qquad \mathrm{app} : (A \rightsquigarrow B) \times A \rightsquigarrow B$$

Every surface construct in every language on the platform lowers to combinations of these six — by an eval scheme declared in `Language` data (Section 4), not by a case in an engine.

2.2 The effect pair: `Op` and `Handle`

Two further constructors carry the effect layer (Sanada's `op` and `handle` (Sanada 2024); Section 5 develops the calculus):

- `Op { dim, name, args, source, lit }` — a named, graded operation request. `source` (`ArrSource`) marks whether the request is a native pure op, a foreign host call (`Foreign(name)` — a module-registry key and capability name, *not* a dimension), a typecheck judgment (`Judgment`), or a dynamic host dispatch (`DynamicForeign`).
- `Handle { body, target, impl_ }` — scope `body` under a handler: `handle body with target = impl_`. Note `target` is a capability name (`String`), not a `dim`-style term slot — grade routing keys off `Op::dim`, not the handle target. The handler is an ordinary arrow value, not a special construct.

2.3 Auxiliary constructors — honest accounting

The `Term` enum in `hnl-engine` carries more than the eight forms above, and this paper does not pretend otherwise. The remaining variants are data-carrying auxiliaries, not new arrow *formers* — they add no new way to build an arrow, they move data the formers need:

- `Const { value }` — a pure value at term level (distinct from `Arr`, which is a constant *arrow*);
- `Iter { fuel, predicate, step, initial }` — *bounded* iteration: a fuel-indexed fold used where `loop` would be unbounded;
- `Child { role }` — project the child occupying a named role of the current AST node (the data-driven counterpart of positional child access; Section 4);
- `Src { cell }` / `Tgt { cell }` — source and target faces of a 2-cell (Section 6);
- `Rewrite { source, target, witness }` — a witnessed rewrite step; the `RewriteWitness` (`Named`/`Compute`/`HandlerHint`/`Cast`) records *why* the step is sound;
- `RecordProj` / `RecordWith` — record field projection and update;
- `Tag { tag, value }` — tagged-value construction;
- `Cobind { grade, body }` — cobinding over a grade and body;
- `Opaque { data }` — a term in *open data form*: a tagged record naming a derived operator (`Path`, `Equiv`, `Whisker`, `CellId`, …) folded out of the closed vocabulary. The kernel carries the record without structural interpretation, and `TermData::to_term` maps any unrecognised *tag* here — which is what makes the `Term` ↔ data conversion total *over tagged records* (a `TermData` without a `__tag__` field is a decode error, not a term): a derived operator that is not (yet) spelled in the closed vocabulary still round-trips.

`Opaque` deserves emphasis because it is the escape hatch that keeps the set closed in practice: an operator the surface calculus derives (`Whisker`, `CellId`, `Univalence`) is represented as *data naming the derivation*, not as a new enum variant. Consumers that do not understand the tag see an opaque record; the law machinery sees the same record the surface forms produce.

2.4 Dimension and axis slots are data

Two slots on the core variants make the higher-dimensional structure of Section 6 possible without new combinators:

- `Arr` and `Op` carry `dim: Box<Term>` (default `Const 1`). A literal `Const n` is a numeric dimension ($\rightsquigarrow_n$); a literal `Const "S"` is a *named* dimension — what the graded-effect surface calls a grade. One slot serves both; a grade is a named dimension. Because `dim` is itself a `Term`, dimension is data inside the calculus, not a static annotation.
- `Seq` carries `axis: Box<Term>` (default `Const 0`). `>>>` is `>>>[0]` — sequential composition of 1-cells; `>>>[k]` composes along the k-th globular axis (Section 6).

`Term::globular_dim` computes the declared dimension structurally — the `dim` slot when it is a literal, else the greatest dimension of the parts. `Term::desugar` is the designated expansion point for residual derived forms: today it is a recursive identity traversal that rebuilds the same constructors and rewrites nothing — actual desugaring happens upstream, in `AstShapeDef`/`transforms` merge data (Section 4).

3 Values

The canonical value type at the `Language`/AST boundary is `serde_json::Value` — the kernel invents no new value type there. `handl-value` (`crates/handl-value`) provides `TaggedValue`, the

tagged representation eval engines use on hot paths, with `From`/`Into` conversion at the boundary. Its variants are the calculus' needs and nothing more: `Null`, `Bool`, `Int(i64)`, `Float`, `Str`, `Bytes`, `Arr`, `Obj`; `Pair` and `EitherLeft`/`EitherRight` as dedicated variants (the `First`, `Choice`, and `Loop` payload shapes); `Arrow`; `Sym` (nominal identity); `Resource` (an RAII-wrapped opaque foreign handle); and `Tensor` (a dense row-major `f64` carrier that engines pass through opaquely to host functions — the `host-tensor` wire form).

3.1 Arrows are serializable data

`TaggedValue::Arrow` is the one computational variant, and its design is a load-bearing claim of the kernel:

> An arrow is **self-contained serializable data**: `{"__arrow__": true, "params": [...], "body_def": ...}` — no closures, no captured lexical environment, no process-local frame references.

At mint time every *free* variable in the body is substituted with its definition-site value (`Ast::instantiate`, `crates/handl-language/src/instantiate.rs`), so the body is a closed term applicable without an environment. The residual `binds` map carries what cannot be substituted — recursion-region binders and values that do not serialize — and `handlers` carries handlers attached by deep-handler attach as explicit data, reinstalled on each application. Binder tracking during substitution is itself data-driven over `AstShapeDef` roles (Section 4), never `kind ==` checks.

Two consequences follow. First, a value can travel — across engines, across a serialization boundary, into a cache — because nothing in it is a pointer into a process. Second, higher-order values need no closure type: `call(f, x)` lowers to a pair quoted through `app`, and an inner function referencing enclosing parameters is a bound-arrow record — a product encoding, not an environment capture.

3.2 Nominal identity

`TaggedValue::Sym` is a first-class nominal identity: a unique `handle` minted from a global counter, a display `name` that is presentation-only, and an optional single-parent `Sym` chain. Equality is identity (`handle` equality) — two syms with equal names are never equal — while the JSON encoding is explicit data that round-trips the handle, so a deserialized sym compares equal to the live sym it came from *within a process*; cross-process identity is unresolved (ADR-0106's stated follow-up). `Sym` is how the kernel provides nominal objects (fresh type constructors, class-like identities) without adding a name-generation construct to the calculus.

4 Language as data

The `handl-language` crate defines the schema (ADR-0044). A `Language` is a struct of `FragmentSet`s plus value tables — pure serializable data end to end: no trait objects, no closures, no `Box<dyn Behavior>`.

4.1 The `Language` value

Table 2: The `Language` schema (`crates/handl-language/src/language.rs`).

Field	Type	What it declares
`tokens`	`FragmentSet<TokenKindDef>`	the token vocabulary and its tooling categories
`lexer_rules`	`FragmentSet<LexerRuleDef>`	how source text becomes tokens (`LexerPattern`s)
`grammar`	`FragmentSet<GrammarRow>`	how tokens become AST — binding powers, child shapes
`ast`	`FragmentSet<AstShapeDef>`	per node kind: payload schema, named child roles, arity, desugar
`eval`	`FragmentSet<EvalSchemeDef>`	how each node kind becomes a `Term`
`types`	`FragmentSet<TypeRuleDef>`	per node kind: `synth`/`check` `Term`s
`transforms`	`FragmentSet<TransformDef>`	`(pass, node_kind, priority)`-keyed rewrite/desugar rules
`caps`	`FragmentSet<CapabilityDecl>`	capability declarations
`options`	`FragmentSet<OptionSchema>`	option schemas
`lints`, `lsp`	`FragmentSet<…>`	lint rules; LSP surface definitions
`ops`, `handlers`, `handlers_by_grade`	`BTreeMap<String, Value>`	evaluated op declarations; handler values; the grade index
`laws`, `rewrites`	`BTreeMap<String, Value>`	declared laws and rewrite rules `source ~> target`
`globals`	`BTreeMap<String, Value>`	ambient global bindings seeded into engine scope
`scheme_version`	`SchemeVersion`	which `Term` combinator version the language targets (`"v2"`)

Two fragment rows deserve detail. `EvalSchemeDef` is `{id, node_kind, scheme: Term, replaces, requires}` — the eval fragment *is* a table of `Term`s, so "how a construct executes" is a data row an engine instantiates directly, with zero Rust arms per construct. `AstShapeDef` carries `children: Option<Vec<ChildRoleSpec>>` — named roles (`role`, `variadic`, `optional`) — plus `literal_children`, `pattern_role`, and an optional `desugar` rule that merge lowers into the `transforms` set (a `TransformDef` under pass `"desugar"`), so a shape and its desugaring cannot drift apart.

4.2 Merge semantics

Composition is `Language::merge`, applied in kit-defined order. The policy (`FragmentSet`, `crates/handl-language/src/fragment_set.rs`) is what makes "language as data" composable rather than merely concatenable:

- **Override vs collision.** A fragment declaring `replaces: [id]` is an explicit override: the replaced ids must be present, cyclic `replaces` chains are rejected, and every override is recorded

in an auditable `override_chain`. Two fragments targeting the same identity key with no `replaces` declared is a *collision* — an error, not a silent winner — except for the four union types (`CapabilityDecl`, `TokenKindDef`, `LexerRuleDef`, `AstShapeDef`), whose merge is idempotent and keeps the existing declaration.
- **Dependencies.** `requires` is validated against the post-merge id set (`MissingRequires`).
- **Atomicity.** `merge` operates on a clone and swaps in only on full success — a failed fragment commits no partial state.
- **Consistency.** `AstShapeDef::desugar` rules are lowered into `transforms` at merge time; when a shape merge drops or replaces a declaration its synthesized `transform.desugar.*` rule is dropped with it, keeping the two sets in sync by construction.

Because a `Language` is a value, assembling one is ordinary programming: the canonical eval-path fragment set is published as YAML data in `kit-prelude-lang`, and `defining-handl` (`crates/defining-handl`) goes further — it produces a `Language` value describing a subset of the host language itself and evaluates it under itself across the eval engines (the Futamura-projection sanity check (Futamura 1999)).

4.3 Engines dispatch on data

The companion rule, enforced across the platform (and stated as a rule of the kernel's own rule files):

> Dispatch on node kind via `Language` data — never a hardcoded `match` on kind strings.

Concretely: child positions are resolved by **named role** — `lang.child_role_index(node.kind, "target")` for shape-level checks, `lang.child_role_index_for(node, role)` for arity-dependent shapes with `optional` roles, `role_name_at` for the inverse, and `literal_child` for the literal-vs-computed distinction (ADR-0081). Pattern-position behavior dispatches on `AstShapeDef::pattern_role`. A language that spells `set/params/body` instead of `assign/args/value` runs on untouched engines — AST layout knowledge lives in the data, not in shape checks inside primitives. What is allowed to be fixed in Rust is only the calculus vocabulary itself: the combinator set, primitive forms, and value shapes.

The staged contract (`staged.rs`) makes the discipline precise: `lex(lang, src)`, `parse(lang, toks)`, `eval(lang, ast)`, and `run = eval ∘ parse ∘ lex` are typed signatures consuming only the previous stage's output plus the `Language` spec — the crate ships them unbound (`LangError::NotImplemented`) because engines are *not* kernel; they are the `handl-lang/engine*` layer that binds against these signatures.

5 Grades and handlers: the GMSAC

The kernel's canonical execution calculus is the **Graded Modal Sanada Arrow Calculus** — three ideas layered on the same `Term`:

1. **Arrows** — the closed set of Section 2.
2. **Grades** — a row of algebraic capabilities annotating an arrow type, read as a *modality* on where the arrow may be interpreted.
3. **Sanada's handlers** — an operation request with an explicit continuation and a scoped interpretation of requests (Sanada 2024).

5.1 Grade rows

A grade row is a *multiset* of grade terms on the arrow type:

```
f : A ~[(Op, Query)]~> B
g : A ~[(Op, Clock, FFI)]~> B
```

A single `Op` contributes exactly one grade — its `dim` slot names the one dimension the request is declared at — and a composite arrow's row is the multiset of its constituent `Op`s' dims. `Op` is the marker grade — "this arrow is an operation request" — and the remaining names are algebra grades. Handlers, in turn, declare the *set* of grades they cover, and a request is interpreted by the innermost handler covering the request's grade. A pure arrow has an empty row; an `Op` adds a grade; installing a handler removes one. Quotienting the multiset to a set gives the usual effect-row subtyping of Bauer–Pretnar (Bauer and Pretnar 2014), restated in arrow terms: `A ~[(Query)]~> B` sits under `A ~[(Query, Clock)]~> B`.

The grade row is not decoration on top of the calculus: it *is* the `dim` slot (Section 2.4). The unified slot is the kernel's answer to a question the earlier draft separated — "where do the grade bracket and the dimension bracket live?" — they are one bracket. A grade is a named dimension; handler routing is dimension-typed; `Term::dim_label` resolves the literal `Const "S"`/`Arr "S"` positions to a routing label, and numeric literals are unnamed dimensions. `ArrSource::Foreign` stays strictly the host-function boundary — a registry key, never a dimension.

In the graded-algebraic-theory reading (Katsumata's parametric effect monads (Katsumata 2014), Kura's graded algebraic theories (Kura 2020), Sanada's category-graded theories (Sanada 2023), and the category-graded monad unification of Orchard–Wadler–Eades (Orchard, Wadler, and Eades III 2020)): sorts are value families, grades are capabilities, an operation is a graded arrow declaration, a `Term` is a term in the free arrow category over the signature, and a model is a `Model` — the trait in `hnl-engine` whose `run(term, incoming, ctx)` interprets terms and whose `rewrite(source, target, witness, ...)` interprets rewrites. Dispatch is two-keyed: the *handler* is selected by grade coverage — the innermost handler whose declared `grades` set covers the op's dimension label — and the *implementation* is then selected by operation name inside that handler's `impls` map. An engine seeing `Op{name: "add"}` resolves the op's grade, checks the carrier's declared grades, routes to the innermost covering handler, and looks up `"add"` in its impls — so the same operation works on every sort that carries it.

5.2 Handlers as elimination

`Term::Op` and `Term::Handle` give the calculus deep, rebindable effects. An `Op` is a request carrying a name, arguments, and its grade row; a `Handle` scopes a body under a handler. The handler is an ordinary arrow value returning a record of operation implementations — not a special form — and installing it is the elimination of the modal grade: bracketing a region with a handler removes the corresponding effect from the row and restores the previous interpretation on exit. Runtime, `Handle` pushes its implementation onto the `ModelCtx` handler stack and `Op` dispatch searches it innermost-last, then falls back to the incoming value's own `impls` field (a tagged value may embed per-dimension op implementations), then to the language-level handler table; `Language::handlers_by_grade` is the compile-time index into that table, kept as data so a language's ambient handlers travel with the language.

Because handlers are values, effect algebras are *packages*, not kernel concepts: a resumable request algebra, a mutation algebra, a fire-and-forget event algebra all lower to `Op`/`Handle` — the runtime

knows only the calculus and never pattern-matches on the algebra's names. User-space algebras are exactly Plotkin–Power operations with Plotkin–Pretnar handlers (G. Plotkin and Power 2003; G. D. Plotkin and Pretnar 2013), carried on arrows per Sanada rather than on monadic computations, and the capability view — effects as what the context *provides* — is the Effekt line (Brachthäuser, Schuster, and Ostermann 2020).

6 Morphisms all the way up

The kernel takes one further step past "arrows compute": *everything* is a morphism, and the same mechanism recurs up the dimensions. This section is the theory level — the calculus structure, not a surface syntax.

6.1 Dimensions and axes

Writing $M[n]\langle A, B\rangle$ for the family of n-morphisms between parallel $(n-1)$-morphisms, dimension 1 is programs and dimension 2 is transformations between them:

```
f, g : A ~[1]~> B            -- programs (1-morphisms between objects)
     : f ~[2]~> g            -- a law / rewrite (2-morphism between 1-morphisms)
     :   ~[3]~>              -- a transformation of laws (3-morphism)
```

Two independent indices organise the calculus. The *dimension* n in $\rightsquigarrow_n$ names what kind of cell is being built — in `Term`, the `dim` slot of `Arr`/`Op`, which is itself a `Term` and therefore data (Section 2.4). The *axis* k in $\ggg_k$ (`Seq{axis, fst, snd}`) names along which boundary cells paste: `>>>` [0] pairs cells across composition of their boundary morphisms, `>>>` [1] stacks them along the morphism direction, and the boundary law is globular — $\mathrm{tgt}_k(a) = \mathrm{src}_k(b)$, enforced for $k \geq 1$; axis-0 composition is unchecked because object boundaries are not part of the value model (`comp_at`, `crates/handl-value/src/lib.rs`). `Term::globular_dim` gates $k < n$, and `cell_comp_axis` computes the *effective* dimension as $\max(\text{declared}, \text{cell_dim}(a), \text{cell_dim}(b))$: `__dim__` n-cell records, `Cell2`, and `dim_arrow` values carry their own height, so `>>>[k]` pastes at any axis $k < n$ — a `Cell{__dim__: 3}` value pastes at axis 2. The cap applies only to degenerate operands: plain values lifted through a declared-dimension-2 scheme are 2-cells, so $k \geq 2$ is a dimension error there — an implementation boundary, stated honestly, not a metatheorem.

The resulting structure is a *weak globular* presentation: higher laws are genuine higher cells (proof-relevant), not strict equalities, with the strict fragment kept as a fast path (below). Cubical Kan structure is deliberately deferred — it would arrive as a cube-diagram fragment over the same `Term`s, not a rewrite of the directed calculus. For the n-categorical background see Leinster's survey (Leinster 2002).

6.2 Laws are 2-morphisms — and therefore data

The structural laws of the calculus are not declarations in a separate rule language; they are 2-morphisms:

```
rightUnit : (f >>> id) ~[2]~> f
assoc     : ((f >>> g) >>> h) ~[2]~> (f >>> (g >>> h))
```

An inhabitant of $M[2]$ is simultaneously a rewrite rule, a proof, and a program transformation; the intended reading is supplied by the consumer — an eval engine, the law machinery, or a rewrite engine — not by a syntactic distinction in the calculus. `Src`/`Tgt` project the boundary faces of

a cell; `Rewrite{source, target, witness}` carries a witnessed step; and derived cell operators (`CellId`, `Whisker`, horizontal/vertical cell composition) fold to `Opaque` tagged records when they are not spelled by a `Seq{axis}` scheme. `Language::laws` and `Language::rewrites` are the value tables these live in — again, data.

Two modules in `handl-language` are the law machinery over this IR:

- `law_inference` — normalizes `Term` graphs to a fixpoint under the CutElim-derived rewrites (`Seq(id,G)  G`, `Seq(G,id)  G`, `Seq(First G, First H)  First(Seq G H)`, `First id  id`) and decides law equality by comparing normal forms — `law_equal`, with `infer_laws` matching a scheme against a catalog of structural law patterns. Its scope is honestly bounded: inference works only for the CutElim-proven structural fragment (the `First` functor laws); semantic laws require explicit axioms, and the module documents that boundary rather than blurring it.
- `quantified_laws` — explicit `forall f g. lhs = rhs` quantified law syntax over `Term`s, checked by instantiation under a representative substitution set: a heuristic strong for schema-level laws, not a completeness proof — the mechanized proof lives in the research prototype's Lean development (handl project 2026a).

`dynamic_handlers` completes the trio: runtime verification of handler implementations against declared laws (`verify_at_runtime`, `VerifiedImplStore`, `dynamic_dispatch`) — the correspondence between unhandled operations and effect rows (Bauer and Pretnar 2014) checked where it can be checked, at runtime.

This is the sense in which the kernel subsumes what other systems keep as four subsystems — a program IR, a rule language, a proof syntax, and a metaprogramming facility: here they are $M[1]$, $M[2]$, $M[2]$, and $M[\geq 3]$ of one mechanism. The same point is what makes the kernel a natural substrate for the org's equality-saturation and discovery work — laws and rewrites are values in the same calculus the engines execute, so a rule bank is data and a derivation is a replayable witness (Tate et al. 2009; Willsey et al. 2021; mlopt project 2026).

7 Why the set is closed

The six combinators (plus `Op`/`Handle`) are the *only* arrow formers the kernel knows, and the closure is a design choice, not an oversight.

7.1 The cost of an open kernel

An open kernel — one where each feature introduces a combinator — accumulates special cases: every combinator must be understood by every consumer of `Term` — each interpreter, the typechecker, the law machinery, the serialiser, the pretty-printer, every handler that interprets a program. Each new combinator multiplies the surface those consumers must cover, and each is a new place for them to disagree. The closed kernel pays the cost of a new feature once, at the boundary where it is translated into the calculus — not in every consumer downstream.

7.2 Minimality evidence

Closure is only defensible if the set is also *minimal* — otherwise a dropped combinator is a latent hole, not a design. The research prototype that precedes this kernel (handl project 2026a) investigated exactly that: at the arrow-graph level the primitive set is *independent* — none of `arr` / composition / `first` / sum routing / `loop` is derivable from the others — and the independence and classification

results are mechanized in Lean 4 (the free-arrow generator argument: e.g. no generator can produce `Sum`/`Prod` output from the wrong primitives; `loop` is independent in a total setting). The sixth combinator, `app`, is not redundant either: it is precisely the closedness — the exponential structure — that makes `ArrowApply` ≅ `Monad` (Hughes 2000; Lindley, Wadler, and Yallop 2010) — the freer-arrow insight that explicit higher-order application recovers the structure opaque continuations lose (Kiselyov and Ishii 2015; VanDomelen et al. 2025).

7.3 How features enter without new combinators

A new construct enters at one of three boundaries, none of which touches the kernel:

1. **Desugaring** — the grammar/`ast` fragments declare a shape whose `desugar` rule or `eval` scheme lowers the construct to existing `Term`s before the kernel ever sees it. Cheapest; the kernel is unaware.
2. **Normalisation** — a transform pass eliminates the construct's encoding between lowering and interpretation. The module system is the worked example in the platform: imports and exports are encoded as ordinary `Op`s and then inlined away, so the kernel never learns that modules existed.
3. **Handler interpretation** — the construct is an `Op` whose meaning a handler supplies. The kernel carries the request opaquely; a `Model` that does not understand it refuses to interpret it — which is *correct*, because interpretation is a model-level fact.

And when even a *scheme* is the wrong level — a derived operator like `Whisker` or `Path` — `Opaque` carries it as data naming the derivation (Section 2).

7.4 When the set should grow

Closed is not frozen. A seventh combinator is justified only when a feature cannot be expressed as a combination of the existing set without losing a property every consumer relies on — that the graph is finite, that feedback is well-founded, that terms serialize. Such cases are rare and belong to an ADR. The default answer is no: find a combination, a desugaring, or a normalisation.

8 What the kernel is not

The kernel's power is what it excludes. Explicitly:

- **Not a language.** There is no user syntax anywhere in the kernel: tokens, grammar, and AST shapes are `Language` data. (`Term` constructors are the calculus vocabulary, not a syntax.)
- **Not an engine.** `lex`/`parse`/`eval` exist only as staged signatures returning `NotImplemented`; every interpreter lives in the `handl-lang/engine*` repos and binds the `Model`/`ModelData` traits.
- **No modules, imports, or files.** These are source-level concerns normalised away before the kernel; `ArrSource::Foreign` is a registry *key*, not a module system.
- **No rules language.** Laws and rewrites are `Term`s at dimension 2 stored in `Language::laws`/`rewrites` — not a parallel DSL.
- **No closures or environments.** Arrow values are self-contained serializable data (Section 3); `instantiate` eliminates free variables at mint.
- **No concrete effect algebras.** Query/command/event-style algebras are package data lowering to `Op`/`Handle`; the kernel dispatches on grade rows only.
- **No concrete type system.** `types` is a fragment set of `synth`/`check` `Term`s; what counts as a type is a `Model` choice.

- **No host composition.** `Plugin/Host/Operation/Behavior/ RuleAcceptor/Contribution`, kits, manifests, and option resolution are the platform layer (`handl-lang/host`), cited here only as the consumer.
- **No extension protocol details.** `handl-ext` provides the shared ABI — JSON over a C boundary: `handl_ext_manifest`, `handl_ext_dispatch`, `handl_ext_free` — so hosts and extensions evolve independently; who loads what is a platform concern.
- **No serialization of authority.** A `Resource/Foreign` value crosses the boundary opaquely; capability tracking is platform policy, not kernel semantics.

9 Implementation status

The kernel is `handl-lang/kernel`, a cargo workspace:

Table 3: Kernel workspace map.

Crate	Contents
`handl-language`	the `Language` schema, `FragmentSet` merge, the `Ast` arena, `staged` dispatch signatures, `law_inference`, `quantified_laws`, `dynamic_handlers`, `instantiate`, `capabilities`, `token_classes`, `verdict`, `version` — plus the `Term/ArrSource/RewriteWitness` re-export from `hnl-engine`
`handl-value`	`TaggedValue`, `Sym`, `TensorValue`, `ForeignResource`, JSON (`serde_json::Value`) conversions
`handl-ext`	the extension ABI (`ExtensionManifest`, `ExtRequest`, `ExtResponse`, `export!`, discovery/loading)
`handl-defining-handl`	self-hosting infrastructure: a `Language` value describing the language subset, plus the self-hosting corpus
`handl-lib-*` floor-library crates (`handl-lib-bytes`, `handl-lib-cmp`, `handl-lib-combinators`, `handl-lib-error`, `handl-lib-mood`, `handl-lib-obj`, `handl-lib-ops`, `handl-lib-prim`, `handl-lib-prim-handl`, `handl-lib-type`)	substrate packages providing the base op/grade tables; they are kernel-workspace crates but depend on `handl-platform` and `handl-plugin-lib-handl` from `handl-lang/host`
`hnl-engine` (dep, `handl-lang/engine`)	`Term`, `TermData`, `Model`, `ModelData`, `ModelCtx`, `CompositeModel`, `AstView/LanguageView`

What is *proven* by the tree: a `Language` value can describe a language and be evaluated under itself — `defining-handl`'s corpus runs across the platform's eval engines, and the Futamura-projection path (compiling the language-as-data interpreter) is exercised (Futamura 1999). What is deferred — full self-hosting of the entire surface language, and a standalone-binary bootstrap — is documented in the crate's own honesty note.

Documented approximations (implementation debt, not design gaps), condensed from the working draft (handl project 2026a):

- `>>>[k]` pastes at any axis $k < n$: `cell_dim` recognises `__dim__` n-cell records (any n), `Cell2`, and `dim_arrow` values, and `cell_comp_axis` computes the effective dimension as $\max(\text{declared}, \text{cell_dim}(a), \text{cell_dim}(b))$ (`crates/handl-value/src/lib.rs`). Only degenerate operands are capped: plain values lifted through a declared-dimension-2 scheme are 2-cells, so $k \geq 2$ is a dimension error there.
- The axis-0 boundary check is exempted: `comp_at` enforces $\text{tgt}_k(a) = \text{src}_k(b)$ for $k \geq 1$ only; 0-boundaries (objects) are not part of the value model, so $\circ_0$ is unchecked horizontal composition.
- The axis slot is an expression position, not a literal-only slot: the tree-walk engine (`hnl-engine-exec-tree-walk`) evaluates a non-literal axis term against the incoming value and errors if the result is not a natural number; the bytecode engine (`hnl-engine-exec-bytecode`) rejects a dynamic axis on 1-cell operands at compile time, because it could resolve to `>>>` — sequential composition — which is not expressible as value-level composition.
- `cell_boundary`-style projection assumes globularity (the globe condition is assumed, not verified).
- Quantified-law checking is substitution-tested, not complete (Section 6).
- A composite of dependent operands flattens to a plain function type at composition — dependent-aware unification exists but a composed body cannot carry its closure across `>>>`.

10 Related work

Arrows. Hughes introduced arrows as a generalisation of monads (Hughes 2000); Lindley, Wadler, and Yallop gave the arrow calculus its minimal form and the idiom/arrow/monad correspondence (Lindley, Wadler, and Yallop 2010, 2011). Our `Term` is that calculus realised as *data* — serializable, dispatched on, rewritten — rather than as a language's type class.

Effects on arrows. Lindley first extended algebraic effects and handlers to idioms and arrows (Lindley 2014); Sanada gave the arrow calculus operations and handlers with operational and denotational semantics (Sanada 2024) and the category-graded theory behind the grade-as-morphism reading (Sanada 2023). The generic algebraic-effects line — Plotkin–Power operations (G. Plotkin and Power 2003), Plotkin–Pretnar handlers (G. D. Plotkin and Pretnar 2013), the Bauer–Pretnar effect system (Bauer and Pretnar 2014), and capability-style effect polymorphism (Brachthäuser, Schuster, and Ostermann 2020) — is the row-and-handler semantics the GMSAC restates on arrows; Moggi's monadic metalanguage (Moggi 1991) is the ancestor everything here descends from.

Grades and indexing. Katsumata's parametric effect monads (Katsumata 2014), Atkey's parameterised notions of computation (Atkey 2009), Kura's graded algebraic theories (Kura 2020), and the category-graded unification (Orchard, Wadler, and Eades III 2020) cover the "grade is data about the computation" axis; the kernel's `dim`-as-`Term` pushes it one step further — the index itself is a term of the calculus.

Explicit vs opaque effects. Kiselyov's freer monads (Kiselyov and Ishii 2015) made continuations inspectable data; freer arrows (VanDomelen et al. 2025) confirm that the explicit input parameter is what keeps fanout and choice *semantically* expressible — the analysis the `Op`-as-request design relies on.

Language as data. The claim that a theory's terms, taken modulo its laws, are themselves an algebraic object is Lindenbaum–Tarski's; Lawvere's functorial semantics made "theory as data, model

as functor" precise (Lawvere 1963). Equality saturation (Tate et al. 2009; Willsey et al. 2021) is the adjacent machinery for which laws-as-data is load-bearing — the org's `eqsat` substrate realises the same cells-as-data pattern in stdlib-only Python, importing none of these crates (mlopt project 2026). The higher-categorical backdrop for the dimension story is standard (Leinster 2002); the Futamura reading of language-as-data is Futamura's (Futamura 1999).

Within the org. The research prototype (`host/research/arrow- calculus-kernel-poc`) carried the mechanized minimality results and the working paper draft this paper tightens (handl project 2026a); the architecture decisions live in the host repo's ADR series — ADR-0044 (language as data), ADR-0052/0104b (the `Term` kernel), ADR-0076/0098 (handlers and grades), ADR-0081 (AST roles), ADR-0103/0106/0119–0120 (opaque data forms, the object/arrow world, the unified dimension slot) (handl project 2026b).

11 Limitations and open questions

- **Metatheory is ahead of the machine where the machine is honest about it.** Whether $M[n]$ as implemented is a family of categories, which coherence laws hold between axes, and which dimension expressions are statically normalizable are open; interchange and higher unitor/associator structure are not yet inhabited cells.
- **The globular boundary is enforced, not verified.** Globularity is assumed where cells are built (Section 9); the value model caps pasting at dimension 2.
- **Law inference is a strict fast path.** It covers the CutElim-proven structural fragment only; general law equality is not decidable here, and quantified-law checking is a heuristic.
- **Dependence is structural, not yet eliminative.** Families indexed by higher cells and dependent elimination (refining a scrutinee's index per match arm) need an equality type the kernel deliberately does not carry.
- **The operational semantics' dimensional lift is partial.** `Op`/ `Handle` ride the unified dimension slot, but how Sanada's semantics lift through arbitrary dimensions is open.
- **Self-hosting is mechanism, not completion** — a subset corpus, not the whole surface language (Section 9).

The claim that survives these limitations is the one this paper set out to check: computation and reasoning about computation — programs, laws, handlers, and the languages that produce all three — are expressible in *one* closed calculus, whose consumers are models and whose languages are data.

References

Atkey, Robert. 2009. "Parameterised Notions of Computation." *Journal of Functional Programming* 19 (3–4): 335–76.

Bauer, Andrej, and Matija Pretnar. 2014. "An Effect System for Algebraic Effects and Handlers." *Logical Methods in Computer Science* 10 (4). https://doi.org/10.2168/LMCS-10(4:9)2014.

Brachthäuser, Jonathan Immanuel, Philipp Schuster, and Klaus Ostermann. 2020. "Effects as Capabilities: Effect Handlers and Lightweight Effect Polymorphism." *Proceedings of the ACM on Programming Languages* 4 (OOPSLA). https://doi.org/10.1145/3428194.

Futamura, Yoshihiko. 1999. "Partial Evaluation of Computation Process — an Approach to a Compiler-Compiler." *Higher-Order and Symbolic Computation* 12 (4): 381–91.

handl project. 2026a. "An n-Dimensional Arrow Calculus for Programmable Computation and Rewriting."

———. 2026b. "The Handl Architecture Decision Record Series."

Hughes, John. 2000. “Generalising Monads to Arrows.” *Science of Computer Programming* 37 (1–3): 67–111. https://doi.org/10.1016/S0167-6423(99)00023-4.

Katsumata, Shin-ya. 2014. “Parametric Effect Monads and Semantics of Effect Systems.” In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM. https://doi.org/10.1145/2535838.2535846.

Kiselyov, Oleg, and Hiromi Ishii. 2015. “Freer Monads, More Extensible Effects.” In *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell*, 94–105. ACM. https://doi.org/10.1145/2804302.2804319.

Kura, Satoshi. 2020. “Graded Algebraic Theories.” In *Foundations of Software Science and Computation Structures (FoSSaCS)*, 12077:401–21. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-030-45231-5_21.

Lawvere, F. William. 1963. “Functorial Semantics of Algebraic Theories.” PhD thesis, Columbia University.

Leinster, Tom. 2002. “A Survey of Definitions of n-Category.” *Theory and Applications of Categories* 10 (1): 1–70.

Lindley, Sam. 2014. “Algebraic Effects and Effect Handlers for Idioms and Arrows.” In *Proceedings of the 10th ACM SIGPLAN Workshop on Generic Programming (WGP)*, 47–58. ACM. https://doi.org/10.1145/2633628.2633636.

Lindley, Sam, Philip Wadler, and Jeremy Yallop. 2010. “The Arrow Calculus.” *Journal of Functional Programming* 20 (1): 51–69. https://doi.org/10.1017/S095679680999027X.

———. 2011. “Idioms Are Oblivious, Arrows Are Meticulous, Monads Are Promiscuous.” *Electronic Notes in Theoretical Computer Science* 229 (5): 97–117. https://doi.org/10.1016/j.entcs.2011.02.018.

mlopt project. 2026. “The Eqsat Substrate: Equality Saturation with Replayable Certificates for Learned Rewrite Games.”

Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55–92. https://doi.org/10.1016/0890-5401(91)90052-4.

Orchard, Dominic, Philip Wadler, and Harley Eades III. 2020. “Unifying Graded and Parameterised Monads.” *Electronic Proceedings in Theoretical Computer Science* 317: 18–38. https://doi.org/10.4204/EPTCS.317.2.

Plotkin, Gordon D., and Matija Pretnar. 2013. “Handling Algebraic Effects.” *Logical Methods in Computer Science* 9 (4). https://doi.org/10.2168/LMCS-9(4:23)2013.

Plotkin, Gordon, and John Power. 2003. “Algebraic Operations and Generic Effects.” *Applied Categorical Structures* 11 (1): 69–94. https://doi.org/10.1023/A:1023064908962.

Sanada, Takahiro. 2023. “Category-Graded Algebraic Theories and Effect Handlers.” *Electronic Notes in Theoretical Informatics and Computer Science* 1. https://doi.org/10.46298/entics.10491.

———. 2024. “Algebraic Effects and Handlers for Arrows.” *Journal of Functional Programming* 34: e8. https://doi.org/10.1017/S0956796824000066.

Tate, Ross, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. “Equality Saturation: A New Approach to Optimization.” In *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 264–76. ACM. https://doi.org/10.1145/1480881.1480915.

VanDomelen, Grant, Gan Shen, Lindsey Kuper, and Yao Li. 2025. “Freer Arrows and Why You Need Them in Haskell.” In *Proceedings of the 18th ACM SIGPLAN International Haskell Symposium (Haskell ’25)*. ACM. https://doi.org/10.1145/3759164.3759352.

Willsey, Max, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. “Egg: Fast and Extensible Equality Saturation.” *Proceedings of the ACM on Programming Languages* 5 (POPL): 1–29. https://doi.org/10.1145/3434304.